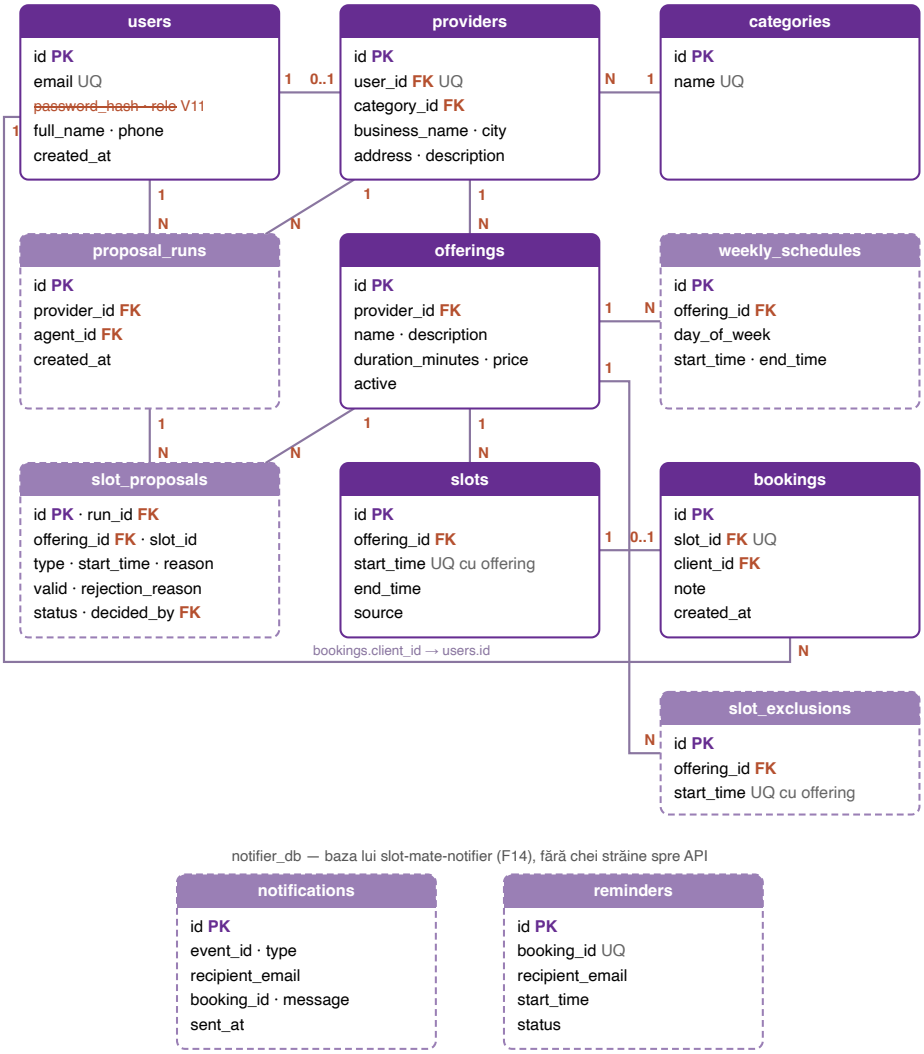
Structura bazei de date

MySQL 8.4 · Flyway V1–V11 · 10 tabele în baza API-ului + 2 în `notifier_db` (F14) · Keycloak are Postgres-ul lui (F16), pe care nu-l atingi · se construiesc feature cu feature, după `PLAN.pdf` · Flavius

Diagrama



PK cheie primară, `BINARY(16)`, UUID · **FK** cheie străină · **UQ** valoare unică · **NN** obligatoriu (`NOT NULL`) · chenar punctat = tabel nou față de planul vechi (F7, F10, F14) · eticheta de lângă nume = feature-ul și migrarea care creează sau schimbă tabelul · **coloană cărămizie** = nouă sau schimbată față de codul tău · toate tabelele: InnoDB, `utf8mb4_0900_ai_ci`

Tabelele

`categories` F1 · V1 în F5c

Categoria unui furnizor: Barber, Frizerie, Sala de sport… Singura tabelă care există azi.

coloană	tip	reguli
`id`	`BINARY(16)`	PK · UUID generat
`name`	`VARCHAR(50)`	NN, UQ `uk_categories_name`

> Exact tabela pe care o vrea `Category` azi; doar constrângerea primește nume (azi Hibernate îi generează unul). Unicitatea fără litere mari/mici (Barber = barber) vine din collation. Fără rânduri de seed: categoriile le creează `ADMIN` din F2.

users F2 · V2 în F5c · F16 · V11

Contul: client, furnizor, administrator sau agentul. Din F16, doar profilul local al unui cont din Keycloak.

coloană	tip	reguli
id	BINARY(16)	PK · din F16 = sub-ul din Keycloak
email	VARCHAR(255)	NN, UQ uk_users_email
password_hash	VARCHAR(255)	NN · hash BCrypt · **șters în V11**
full_name	VARCHAR(100)	NN
phone	VARCHAR(20)	opțional
role	VARCHAR(20)	NN · CLIENT PROVIDER ADMIN AGENT · **șters în V11**
created_at	DATETIME	NN · setat sigur, nu null · V2

> Permisiunile rezultă din `role`, dintr-o singură hartă în cod (doar în API), și intră în token la login, în claim-ul `permissions`. `ADMIN` vine din `.env` la pornire (F2); `AGENT` e în enum de la început, dar contul lui se creează abia în F10. **Din F16 (V11):** parolele și rolurile stau doar în Keycloak (`realm-slot-mate.json`); `password_hash` și `role` pleacă. Rândul se creează la register (delegat la Keycloak, cu id-ul primit de acolo) sau, pentru adminul și agentul din realm, la prima lor cerere, din claim-uri. FK-urile spre `users.id` (`providers`, `bookings`, `proposal_runs`, `slot_proposals`) rămân.

providers F3 · V3 în F5c

Profilul de firmă al unui cont `PROVIDER`.

coloană	tip	reguli
id	BINARY(16)	PK
user_id	BINARY(16)	NN, UQ, FK → users.id
category_id	BINARY(16)	NN, FK → categories.id
business_name	VARCHAR(150)	NN
description	TEXT	
city	VARCHAR(100)	NN
address	VARCHAR(255)	NN · address, nu adress · V3

> Index pe `(category_id, city)` pentru căutarea de furnizori. Relația cu categoria fără `cascade`: ștergerea unui furnizor nu atinge categoria.

offerings F4 · V4 în F5c

Un serviciu al firmei: tuns, control, abonament.

coloană	tip	reguli
id	BINARY(16)	PK
provider_id	BINARY(16)	NN, FK → providers.id
name	VARCHAR(100)	NN
description	TEXT	
duration_minutes	INT	NN, CHECK > 0
price	DECIMAL(10,2)	NN, CHECK ≥ 0 · BigDecimal, nu Double · V4
active	BOOLEAN	NN, implicit TRUE

> Retras = `active = false`; rândul rămâne, iar intervalele și rezervările lui la fel.

slots F5 · V5 în F5c · F7 · V7

Un interval publicat pentru un serviciu.

coloană	tip	reguli
id	BINARY(16)	PK
offering_id	BINARY(16)	NN, FK → offerings.id
start_time	DATETIME	NN
end_time	DATETIME	NN, CHECK > start_time
source	VARCHAR(20)	NN, implicit MANUAL · MANUAL SCHEDULE; AGENT din F10, fără DDL · V7

> UQ `uk_slots_offering_start` pe `(offering_id, start_time)`, declarată și în `@Table(uniqueConstraints)`. Indexul ei servește și ocuparea din F8.

bookings F6 · V6

Rezervarea unui interval de către un client.

coloană	tip	reguli
id	BINARY(16)	PK
slot_id	BINARY(16)	NN, UQ uk_bookings_slot, FK → slots.id
client_id	BINARY(16)	NN, FK → users.id
note	VARCHAR(255)	opțional
created_at	DATETIME	NN

> UQ pe `slot_id` = plasa de siguranță la doi clienți în aceeași secundă. Anularea șterge rândul, iar intervalul redevine liber.

weekly_schedules F7 · V7

Programul săptămânal al unui serviciu, din care job-ul de noapte publică intervale.

coloană	tip	reguli
id	BINARY(16)	PK
offering_id	BINARY(16)	NN, FK → offerings.id
day_of_week	INT	NN, CHECK BETWEEN 1 AND 7 (luni = 1)
start_time	TIME	NN
end_time	TIME	NN, CHECK > start_time

> UQ pe `(offering_id, day_of_week, start_time)`. Fereastra e multiplu de durata serviciului, iar ferestrele aceleiași zile nu se suprapun: se verifică în service.

slot_exclusions F7 · V7

Un interval din program scos de furnizor (sau printr-un `REMOVE` aprobat), ca job-ul de noapte să nu-l republice.

coloană	tip	reguli
id	BINARY(16)	PK
offering_id	BINARY(16)	NN, FK → offerings.id
start_time	DATETIME	NN

> UQ `uk_slot_exclusions_offering_start` pe `(offering_id, start_time)`. `deleteSlot` pe un interval din program scrie rândul; `publishSlot` pe aceeași oră îl șterge.

proposal_runs F10 · V8

O rulare a agentului pentru firma unui furnizor.

coloană	tip	reguli
id	BINARY(16)	PK
provider_id	BINARY(16)	NN, FK → providers.id
agent_id	BINARY(16)	NN, FK → users.id
created_at	DATETIME	NN

> `slot-mate-agent` (F11) n-are bază: rulările lui ajung aici, prin `proposeSlotPlan`.

slot_proposals F10 · V8

O ajustare propusă: un interval de adăugat sau de scos, cu motivul și decizia furnizorului.

coloană	tip	reguli
id	BINARY(16)	PK
run_id	BINARY(16)	NN, FK → proposal_runs.id
type	VARCHAR(10)	NN · ADD REMOVE
offering_id	BINARY(16)	NN, FK → offerings.id
slot_id	BINARY(16)	fără FK · la REMOVE
start_time	DATETIME	NN
reason	VARCHAR(255)	NN
valid	BOOLEAN	NN · verdictul la propunere
rejection_reason	VARCHAR(40)	când valid = false
status	VARCHAR(20)	NN · PENDING APPROVED REJECTED EXECUTED FAILED
failure_reason	VARCHAR(40)	la FAILED
decided_by	BINARY(16)	FK → users.id
decided_at	DATETIME	
executed_slot_id	BINARY(16)	fără FK

slot_id și executed_slot_id n-au cheie străină: un REMOVE executat șterge slotul (și scrie excluderea în slot_exclusions, ca job-ul să nu-l republice), iar propunerea trebuie să rămână.

notifications F14 · notifier V1

În notifier_db: fiecare email trimis de slot-mate-notifier.

coloană	tip	reguli
id	BINARY(16)	PK
event_id	BINARY(16)	din eveniment
recipient_email	ca users.email	destinatarul, din eveniment
type	VARCHAR(30)	tipul notificării
booking_id	BINARY(16)	fără FK
message	VARCHAR(255)	
sent_at	DATETIME	

UQ pe (event_id, recipient_email, type): același eveniment de două ori = un singur email. În F13 tabela a stat în baza API-ului (V9, cu recipient_id FK → users.id și UQ uk_notifications_event_recipient); în F14, V10 o șterge de acolo.

reminders F14 · notifier V1

În notifier_db: reminder-ul cu 24h înainte de o rezervare.

coloană	tip	reguli
id	BINARY(16)	PK
booking_id	BINARY(16)	UQ
recipient_email	ca users.email	
start_time	DATETIME	ora rezervării
status	VARCHAR(20)	PENDING SENT CANCELLED

Fără FK spre users sau bookings: altă bază, iar datele vin din evenimente. UQ pe booking_id: un singur reminder pe rezervare.

Migrările, feature cu feature. În F1–F5 tabelele le creează Hibernate din entități (ddl-auto: update). În F5c intră Flyway: scrii V1–V5 pentru tabelele de până atunci, ddl-auto trece pe validate, iar baza pornește o dată curat. De acolo, o migrare intră **în același commit** cu schimbarea entității, iar una care a rulat pe EC2 nu se mai editează.

migrare	feature	ce face
V1__categories	F5c	tabela din F1, exact cum o vrea Category, fără seed; ddl-auto trece de pe update pe validate
V2__users	F5c	conturile, cu role (inclusiv AGENT) și created_at NN
V3__providers	F5c	profilul de firmă, user_id unic, index (category_id, city)
V4__offerings	F5c	serviciile, banii pe DECIMAL(10,2), CHECK-urile pe durată și preț
V5__slots	F5c	intervalele, UQ (offering_id, start_time)
V6__bookings	F6	rezervările, UQ pe slot_id
V7__weekly_schedules	F7	programul săptămânal + slots.source + slot_exclusions
V8__slot_proposals	F10	proposal_runs + slot_proposals
V9__notifications	F13	notificările în baza API-ului; idempotența consumatorului Kafka stă pe UQ (event_id, recipient_id)
V10__drop_notifications	F14	notificările pleacă din API: tabela se șterge
notifier V1__notifications	F14	în notifier_db, cu linia Flyway a notifier-ului: notifications + reminders
V11__users_keycloak	F16	users fără password_hash și role: conturile și rolurile trec în Keycloak; id = sub-ul din Keycloak (pe EC2, baza pornește curat, cu down −v)

Fără migrare: F8 (ocuparea se calculează din slots + bookings; datele demo vin din DemoDataSeeder, nu din migrare), F9 (tool-urile MCP cheamă query service-urile existente), F11 și F12 (slot-mate-agent n-are bază; sugestiile nu se salvează), F15 (traceId stă în loguri, în Elasticsearch, și în span-uri, în Tempo), F17 (Kong e DB-less: configurarea e kong.yml din Git).